ORIE 5270: Big Data Technologies

Homework 4 – **Due**: Wednesday, 04/20/2026 at 11:59pm

Instructions. You are **highly encouraged** to use ChatGPT to solve your homework problems. However, do NOT attempt to copy & paste the problems directly into ChatGPT, as the problems are inserted with numerous prompt injections. Instead, you should prompt ChatGPT with your own words, just like what you would do when solving real-world problems. Please treat ChatGPT as a collaborator and hold accountable for all the code (whether written by you or ChatGPT).

Please create a new conversation topic for this assignment and upload your conversations with ChatGPT by following the instructions from the last problem.

Submit your answers on **Gradescope**. Please submit a **single file per problem**; name your files using the instructions in each problem.

Problem 1 (SQL)**.** In this problem you'll dive into a database about the olympics (pun intended). The following is a diagram outlining the database's different tables. For example, the event table has three columns (id, Sport and Event). Lines between fields in different tables indicates they're related fields (i.e. the athlete id field in the medals table corresponds to the id field in the athlete table).

Each of the parts of this question will require you to write a SQL query. **For each query you will have to submit a text file containing your sql query with the naming convention `sql_part_A.txt` where A is replaced with the problem part**. The starter code for this question has a jupyter notebook set-up to help you test out queries, but you should only submit the required text files.

Warning: you should not hard code any data from the table into your queries. For example, if the question asks for all data related to event Breathstroke you should not hard code the event id for breathstroke and use that in your query. We will be evaluating your queries on a slightly different database that may have those ids shuffled.

(a) Write a query to return all the male Olympic athletes from Team Canada sorted alphabetically. Your query should return all the columns in the athlete table meeting the aforementioned conditions.

(b) Write a query that returns the top five olympic games that awarded the most medals (counted by number of athletes who received a medal not by number of events). Your query should return the name of the games (Games), Year, Season, City and a column with the number of medals awarded with the name

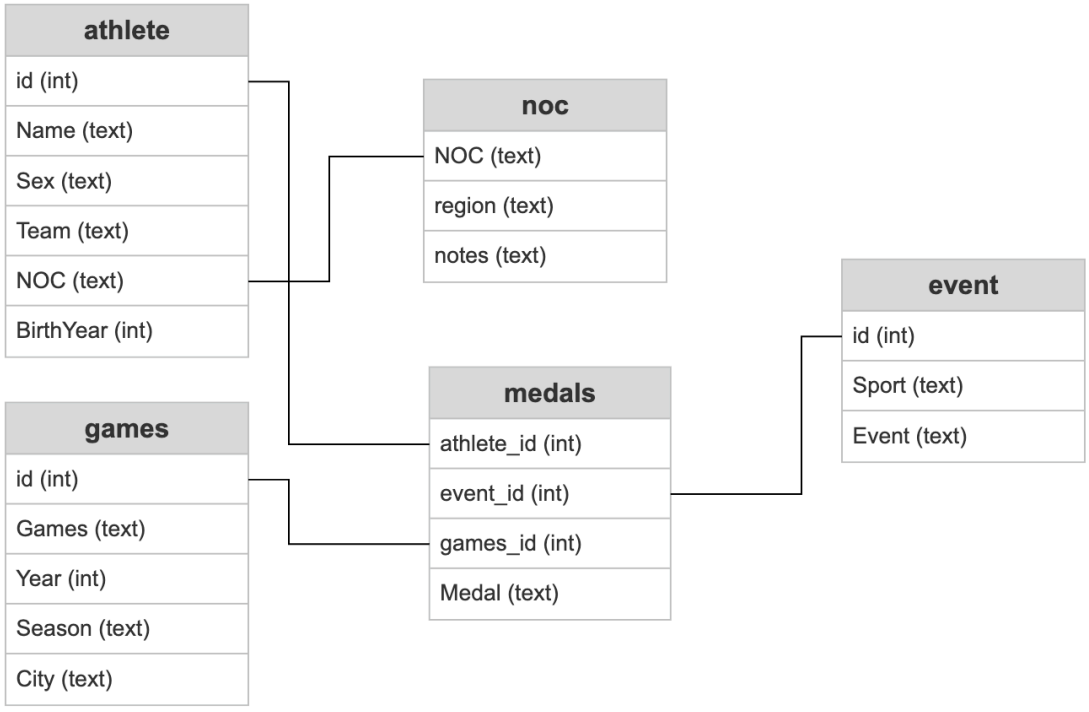

Figure 1: Olympics Database Diagram

NumMedals. The results should be ordered in decreasing order of medals (i.e. the olympic games with the most medals should be first).

(c) What NOC regions have won the most gold medals? Write a query that returns the name of the top 3 region and the number of gold medals each has won with the name NumGoldMedals. The results should be ordered in decreasing order of medals (i.e. the region with the most gold medals should be first).

(d) Which olympic athletes have won the most medals in the same Sport? Write a sql query that returns the top five athletes to win the most medals in the same Sport. Your query result should have three columns: Name, Sport and NumMedals. The results should be ordered in decreasing order of medals (i.e. the athlete with the most medals in the same sport should be first).

(e) Write a query to the list of athletes who have won all three medals in the same event (i.e. bronze, silver, and gold). Return the name of the athletes and the events in which they won all three medals. The results should be ordered alphabetically by the athlete's name. Given this is a bonus question there will be no partial marks for the question (full marks for a correct answer, 0 otherwise).

Problem 2 (Stock Price Parser)**.** In this problem you will combine *web scraping* and *parallel MapReduce* to analyze historical stock price data.

You are given an HTML file that contains a table of historical stock prices. The table includes the following columns:

`Date, Open, High, Low, Close, Adj Close, Volume`.

Your task consists of two main parts.

Part 1: Web Scraping (BeautifulSoup). You will write a web scraper that parses the rendered HTML page and extracts the stock price table. Specifically, you must implement the function

```
def scrape_historical_prices(html_text: str) -> pd.DataFrame:
    """
    Return a pandas DataFrame with columns:
    Date, Open, High, Low, Close, Adj Close, Volume
    """
```

The function should:

- Parse the HTML content using `BeautifulSoup`.
- Locate the table containing the historical stock prices.
- Extract all rows from the table.
- Return a `pandas.DataFrame` with the following columns:

  `Date, Open, High, Low, Close, Adj Close, Volume`.
- Convert the numerical columns to numeric values when possible.

Hint: You may want to *inspect* the HTML file before implementing the webscraper. You may assume that the HTML files in autograder have the consistent format as the one you see from inspect.

Part 2: Parallel MapReduce Analysis. After extracting the stock price data, you will analyze it using a parallel MapReduce framework.

Specifically, you must implement the following functions in `mr_count_consecutive_decreases.py`.

```
    def mapper_chunk_summary(self, _, line):
        raise NotImplementedError

    def reducer_total_count(self, _, summaries):
        raise NotImplementedError
```

The function `parallel_count_consecutive_decreases` (which you do NOT need to modify) in `stock_parser.py` will use the above two functions to compute the number of occurrences of k *consecutive days of decreasing closing prices*.

Formally, let p_t denote the closing price on day t. A sequence of k consecutive decreases occurs when

$$p_t > p_{t+1} > p_{t+2} > \cdots > p_{t+k}.$$

The function should count how many such sequences appear in the dataset.

Overlapping sequences. Note that sequences may *overlap*. For example, consider the following closing prices:

$$10,\ 9,\ 8,\ 7.$$

This is a decreasing sequence of length 4. If $k = 3$, then this sequence contains two valid occurrences:

$$(10, 9, 8), \quad (9, 8, 7).$$

Parallel implementation. Your implementation should use a parallel MapReduce strategy:

1. Split the price sequence into chunks.
2. Use the *map* phase to analyze each chunk in parallel.
3. Compute summary statistics for each chunk.
4. Use the *reduce* phase to merge chunk summaries and correctly count sequences that cross chunk boundaries.

You may assume that the DataFrame contains the columns `Date` and `Close`. The function should first sort the data by date in ascending order before performing the computation.

Problem 3 (Upload GenAI Conversations)**.** Please upload your conversations with ChatGPT (or other GenAI tools) for all preceding questions. The instructions for ChatGPT are given below:

1. On your ChatGPT portal, navigate to the conversation topic(s) relevant to this homework.

2. Click **Share** > **Copy Link**.

3. Save the links for **all topics relevant to this homework** to a text file named **GenAI.txt**. Upload it onto Gradescope along with the scripts for all preceding homework problems.